

HAWK – University of Applied Sciences and Arts

Faculty of Engineering and Health

Professorship for Robotics and Embedded Systems

Autonomous Line-Following Robot Using Camera-Based Lane Detection

Project Report – Autonomous Systems

Winter Semester 2024/25

**Sadri Ghamshi, Ashkan
Mejri, Edem
Benfarhat, Mohamed Wajih**

Supervisor: Prof. Dr. rer. nat. Thomas Linkugel

February 22, 2026

Abstract

This report documents a lane-following system we built for the TurtleBot3 Waffle robot as part of the Autonomous Systems course at HAWK University. The robot uses its camera to detect two white lines on the floor and drives between them. We implemented the full pipeline from camera calibration and bird's-eye view transformation through LiDAR overlay and line detection to closed-loop velocity control. In simulation (Gazebo), a PID controller keeps the robot centered with high accuracy. On the real robot, we switched to a simpler proportional controller with multi-height scanning to deal with noise and lighting issues. Both modes complete full laps autonomously, and a three-tier fallback strategy handles curves where one line goes out of view.

Contents

1	Introduction	5
1.1	Background and Project Environment	5
1.2	Motivation	5
1.3	Objectives	6
1.4	Scope	6
1.5	Methodology	6
1.6	Team Organization	7
1.7	Report Structure	7
2	System Overview	9
2.1	Hardware Platform	9
2.2	Software Framework	9
2.3	System Architecture	10
2.3.1	Simulation Pipeline	10
2.3.2	Real Robot Pipeline	10
2.3.3	ROS2 Topics	10
2.4	Configuration	11
2.5	Development and Debugging Tools	11
3	State of the Art	13
3.1	Lane Detection Methods	13
3.2	Perspective Transformation	14
3.3	Camera Calibration	14
3.4	Control Strategies	14
3.5	Multi-Height Scanning	15
3.6	Summary	15
4	Requirements Analysis	17
4.1	Functional Requirements	17
4.2	Non-Functional Requirements	18
4.3	Use Cases	18
4.4	Performance Metrics	19
4.5	Requirements Validation	19
4.6	Traceability	20

5	Concept Development	21
5.1	Detection: Thresholding vs. Edge Detection	21
5.2	Simulation: PID with Curvature-Aware Speed	21
5.3	Real Robot: P-Only with Multi-Height Scanning	21
5.4	Fallback Strategy	22
6	Implementation	23
6.1	Camera Calibration and Undistortion	23
6.2	Bird's-Eye View and LiDAR Overlay	23
6.3	Line Detection — Simulation	23
6.4	Line Detection — Real Robot	24
6.5	Controller Design	24
6.5.1	Simulation (PID)	24
6.5.2	Real Robot (P-Only)	24
7	Testing and Evaluation	26
7.1	Simulation Results	26
7.2	Real Robot Results	26
7.3	Performance Summary	27
7.4	Issues	27
8	Conclusion	28
8.1	What We Built	28
8.2	What Worked Well	28
8.3	What Did Not Go So Well	28
8.4	Future Improvements	28
A	Appendix	30
A.1	Configuration Files	30
A.1.1	Real Robot Parameters	30
A.1.2	Simulation Homography Points	31
A.2	Project Repository Structure	31

List of Figures

2.1	Simulation pipeline. Each block also publishes its output to a ROS topic for debugging.	10
6.1	Multi-height scanning: six lines from far (blue) to near (red).	24
7.1	Detection mode transitions through a curve.	26

List of Tables

1.1	Division of work among team members.	7
1.2	Page allocation per chapter (excluding front matter, bibliography, and appendix).	7
2.1	ROS 2 topics in simulation mode.	11
2.2	ROS 2 topics in real robot mode.	11
3.1	Comparison of detection approaches for indoor line following.	15
4.1	Requirements traceability.	20
6.1	Controller comparison.	25
7.1	Performance comparison.	27

Chapter 1

Introduction

Author: Mejri, Edem

1.1 Background and Project Environment

Autonomous mobile robots have become a major topic in both academic research and industrial applications over the past decade [1]. From warehouse logistics to agricultural automation, the ability for a robot to navigate its environment without human intervention is increasingly relevant. One of the most fundamental tasks in this field is line following — the ability to detect and track a path marked on the ground, which serves as a starting point for more complex navigation behaviors.

This project was developed within the *Autonome Systeme* (Autonomous Systems) module at HAWK University of Applied Sciences and Arts in Göttingen, supervised by Prof. Dr. rer. nat. Thomas Linkugel. The course provides hands-on experience with robotics using the TurtleBot3 platform and ROS2, and the semester project requires students to build a working vision-based line follower from scratch.

1.2 Motivation

At first glance, line following might seem like a trivial task. In simulation, where lighting is perfect and the track has clean edges, it actually is fairly straightforward. However, once you move to a real robot in a real lab, things change dramatically. The camera picks up reflections from the blue floor, the white tape lines are not perfectly straight, ambient lighting creates uneven brightness, and the robot vibrates as it moves. All of these factors make the problem much harder than it appears in theory.

This gap between simulation and reality — often called the “sim-to-real gap” in robotics — is one of the central themes of this project. Algorithms that perform flawlessly in Gazebo may need fundamental redesign when deployed on physical hardware. Understanding when a simulation result will transfer and when it will not is a practical skill that this project aims to develop.

The central question we set out to answer is: how can we build a lane detection and following system that works reliably in both simulation and on real hardware, and degrades gracefully when it cannot see the full track?

1.3 Objectives

The project addresses seven mandatory tasks defined in the course specification:

- T1** Calibrate the intrinsic and extrinsic camera parameters using a checkerboard pattern.
- T2** Publish the rectified (undistorted) camera image on a ROS2 topic.
- T3** Determine the homography matrix for the perspective transformation.
- T4** Publish the bird’s-eye view image.
- T5** Determine the transformation between the bird’s-eye view and the LiDAR point cloud.
- T6** Publish the bird’s-eye view with LiDAR points overlaid.
- T7** Implement a lane detection algorithm that follows the lane and interpolates when one line is missing.

Tasks T1 through T6 build on each other in a pipeline: calibration enables undistortion, which enables accurate homography, which enables the bird’s-eye view, which provides the coordinate frame for the LiDAR overlay. Task T7 then uses this pipeline as input for the actual driving behavior. This sequential dependency meant that a bug in an early stage (for example, incorrect homography points) would break everything downstream, so we invested significant effort in validating each stage independently before moving to the next.

1.4 Scope

We focused exclusively on classical computer vision techniques. Deep learning, obstacle avoidance, trajectory visualization in RViz, and map building through image stitching were not part of the scope, although some of these are listed as optional bonus tasks in the specification. The system was developed for indoor use on a predefined track with white lines on a dark surface.

One of the most interesting aspects of this project turned out to be the gap between simulation and reality. In Gazebo, the lighting is uniform, the floor texture is perfectly flat, and the camera produces noise-free images. On the real robot in our lab, the blue floor creates specular reflections, the white tape is not perfectly smooth, and the camera introduces noise from vibration and varying ambient light. This sim-to-real gap ended up driving many of our design decisions, as we discuss in the later chapters.

1.5 Methodology

Our development process followed a roughly linear path through the seven mandatory tasks, but with significant iteration. We started in simulation, where development cycles are fast because you can reset the robot instantly and the results are reproducible. Once each module worked in Gazebo, we moved to the real robot, which almost always required adjustments. Some

adjustments were minor (changing a threshold value), while others were fundamental (replacing the entire controller architecture).

We used Git for version control and organized the code as a single ROS2 package. All tunable parameters were externalized into YAML configuration files early on, which turned out to be one of the most impactful decisions for productivity — during the final testing phase, we were changing parameters and re-running within seconds instead of editing code and rebuilding.

1.6 Team Organization

This project was carried out by a team of three students. To make it clear who did what, each chapter has an author tag in the header, and the division of work is summarized in Table 1.1. Table 1.2 shows how the 25 pages of content are distributed.

Table 1.1: Division of work among team members.

Team Member	Responsibility
Mejri, Edem	Introduction, system overview
Sadri Ghamshi, Ashkan	Concept development, implementation (all modules), controller design, testing and evaluation, conclusion
Benfarhat, Mohamed Wajih	State of the art research, requirements analysis

Table 1.2: Page allocation per chapter (excluding front matter, bibliography, and appendix).

Ch.	Title	Author	Pages
1	Introduction	Mejri, Edem	4
2	System Overview	Mejri, Edem	4
3	State of the Art	Benfarhat, M. W.	4
4	Requirements Analysis	Benfarhat, M. W.	4
5	Concept Development	Sadri Ghamshi, Ashkan	2
6	Implementation	Sadri Ghamshi, Ashkan	3
7	Testing and Evaluation	Sadri Ghamshi, Ashkan	2
8	Conclusion	Sadri Ghamshi, Ashkan	2
Mejri, Edem — Subtotal			8
Benfarhat, Mohamed Wajih — Subtotal			8
Sadri Ghamshi, Ashkan — Subtotal			9
Report Total (excl. Appendix)			25

1.7 Report Structure

The rest of this report follows the standard structure for engineering project reports. Chapter 2 gives an overview of the TurtleBot3 hardware and the ROS2 software stack we used. Chapter 3

reviews what methods exist in the literature for line following. Chapter 4 breaks down the seven tasks into formal requirements with test criteria. Chapter 5 explains why we chose the approaches we did. Chapter 6 goes into the actual code and algorithms. Chapter 7 shows how well everything works in practice. Chapter 8 wraps things up with lessons learned and future improvements.

Chapter 2

System Overview

Author: Mejri, Edem

This chapter describes the hardware and software we worked with, and how the overall system is put together.

2.1 Hardware Platform

We used the TurtleBot3 Waffle by ROBOTIS [2], which is a small differential-drive robot commonly used in ROS2 education. It has a 360-degree LDS-02 LiDAR and an IMX708 RGB camera. The camera is mounted at the front of the robot pointing downward at the floor, but due to the way the mount is designed, it actually sits upside-down. We fix this in software by rotating the image 180 degrees before doing anything else.

The track in our lab at HAWK is a blue floor with white tape forming an oval loop with straight parts and 90-degree corners. The lane is about 50 cm wide between the two white lines, and each line is roughly 5 cm wide. The lab environment presents several challenges that are not present in simulation: fluorescent ceiling lights create uneven illumination across the track, the glossy blue floor produces specular reflections that can look like white lines to the camera, and nearby objects (a yellow door, equipment boxes, cables on the floor) occasionally enter the camera’s field of view and confuse the detection algorithm.

The robot’s onboard computer is a Raspberry Pi 4 with 4 GB of RAM, running Ubuntu 24.04. This limits the computational budget for image processing — algorithms that run comfortably on a lab workstation might be too slow on the Pi. We kept this constraint in mind throughout the design, favoring simple and fast operations over computationally expensive ones.

2.2 Software Framework

Everything runs on ROS2 Jazzy Jalisco [3] with Ubuntu 24.04 and Python 3.12. We used OpenCV [4] for all the image processing, and all tunable parameters (thresholds, controller gains, etc.) are stored in YAML config files so we could change things quickly during testing without editing code.

For simulation, we used Gazebo Harmonic, which gives us a physics engine and emulated sensors. We built a custom world file that replicates our lab track with the same oval shape and

corner geometry.

2.3 System Architecture

The codebase lives in a single ROS2 package called `line_follower_pkg`. Inside, there are two separate pipelines: one for simulation and one for the real robot. We ended up keeping them separate because the environments are so different that trying to share code between them caused more problems than it solved.

2.3.1 Simulation Pipeline

The simulation node (`line_follower_node.py`) subscribes to the camera and LiDAR topics and runs a five-stage pipeline on every frame. Figure 2.1 shows the flow.

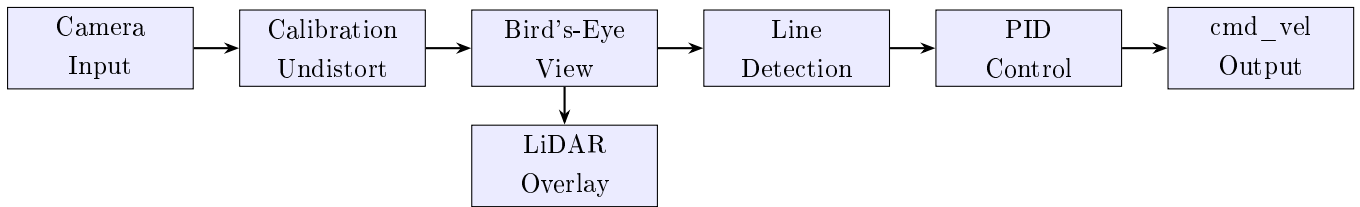


Figure 2.1: Simulation pipeline. Each block also publishes its output to a ROS topic for debugging.

The raw camera image first gets undistorted using precomputed calibration maps. Then a homography matrix $\mathbf{H}$ warps it into a top-down bird's-eye view. LiDAR points get projected into this view as colored dots. The line detector finds the lane boundaries and computes a center offset, which the PID controller turns into velocity commands. Four separate debug topics let us watch each stage in `rqt_image_view`.

2.3.2 Real Robot Pipeline

The real robot node (`real_robot_line_follower_node.py`) is simpler because we skip the bird's-eye view and LiDAR overlay entirely. The camera image gets rotated 180 degrees, then the bottom half is cropped (that is where the floor is), resized to 640×480 , and turned into a binary black-and-white image. Six horizontal scan lines at different heights find the left and right lane edges. A proportional controller computes the steering based on how far off-center we are.

2.3.3 ROS 2 Topics

Table 2.1 and Table 2.2 list the topics used in each mode. The simulation has more topics because of the extra pipeline stages.

Table 2.1: ROS2 topics in simulation mode.

Topic	Type	Description
/camera/image_raw	sensor_msgs/Image	Raw camera
/camera/rectified_image	sensor_msgs/Image	Undistorted
/camera/birdseye_image	sensor_msgs/Image	Top-down view
/camera/birdseye_with_lidar	sensor_msgs/Image	Top-down + LiDAR
/camera/line_detection	sensor_msgs/Image	Debug overlay
/cmd_vel	TwistStamped	Velocity output
/scan	LaserScan	LiDAR input

Table 2.2: ROS2 topics in real robot mode.

Topic	Type	Description
/camera/image_raw	sensor_msgs/Image	IMX708 camera
/camera/line_detection	sensor_msgs/Image	Debug overlay
/cmd_vel	TwistStamped	Velocity output

2.4 Configuration

All parameters live in YAML files under `config/`. There is one for homography points, one for the real robot’s homography, one for controller gains and speeds, and one for camera calibration (though as we discuss later, the calibration file did not make it into the final repository). This turned out to be really important during testing — we must have changed the angular gain alone about 30 times before finding a value that worked well on the real robot.

The configuration approach also makes it easy to switch between environments. For example, the detection threshold is 110 in simulation but 200 on the real robot, and the homography source points are completely different because the camera mounting angles differ. By keeping all of this in config files rather than hardcoding it, we could hand someone the robot and they could recalibrate for a different track layout without reading any Python code.

2.5 Development and Debugging Tools

Beyond the main pipeline, we built several supporting tools that proved essential during development. The camera calibration tool (`calibrate_camera.py`) provides a live view where you hold up a checkerboard, capture frames, and compute intrinsic parameters interactively. The homography calibration tool (`calibrate_homography.py`) lets you drag four points on a live image and immediately see the bird’s-eye view update. Both tools save their output to YAML files that the main pipeline reads at startup.

For debugging during runs, every pipeline stage publishes its output to a separate ROS2 topic. This means you can open `rqt_image_view` and watch what each module sees in real time — the raw camera image, the undistorted version, the bird’s-eye view, the LiDAR overlay,

and the final detection overlay with scan lines and lane center markers. On the real robot, the detection debug topic additionally saves images to disk whenever the detection mode changes, which gives us a frame-by-frame record of how the system handled each curve. We used these saved images extensively for the analysis in Chapter 7.

Chapter 3

State of the Art

Author: Benfarhat, Mohamed Wajih

Before jumping into the design, it makes sense to look at what approaches already exist for the kind of problem we are trying to solve. This chapter surveys the relevant literature on lane detection, perspective transformation, camera calibration, and control strategies.

3.1 Lane Detection Methods

In the lane detection space, there are broadly two families of approaches: classical computer vision and deep learning.

Classical methods typically involve converting the image to grayscale or another color space, running an edge detector like Canny, masking out a region of interest, and then using the Hough Transform to find straight lines [5]. The Hough Transform works by mapping each edge pixel into a parameter space where lines appear as peaks. The standard variant finds only straight lines, while the probabilistic Hough Transform can handle line segments, which is more practical for real images. This pipeline works reasonably well for highway lanes where the markings are long and clear, but in our indoor setting, the Canny edge detector picks up edges from everything — floor seams, nearby objects, even shadows cast by the robot itself. We actually tried this approach early on during a test session and the output was so cluttered with false edges that the Hough line detector kept finding lines that had nothing to do with the track. Filtering out the bad detections was possible but required a lot of heuristic rules that made the code fragile.

Binary thresholding is the other classical option. The idea is straightforward: if the white lines are much brighter than the floor, just set a brightness cutoff and everything above it is “line,” everything below is “not line.” This is fast and surprisingly effective when the environment is controlled [5]. The downside is that it breaks if the lighting changes significantly. More adaptive versions like Otsu’s method compute the threshold automatically from the image histogram by finding the value that minimizes intra-class variance. Local adaptive thresholding goes even further by computing different thresholds for different regions of the image, which handles uneven illumination.

On the deep learning side, convolutional neural networks trained on large lane detection datasets have shown impressive results on driving benchmarks [5]. These approaches treat lane detection as a segmentation task, outputting pixel-wise lane masks or learning spatial relation-

ships across the image. They are powerful but they need a lot of labeled training data, they are computationally heavy (often requiring a GPU), and a model trained on highway images would not help us detect white tape on a blue floor. Given the TurtleBot3’s limited onboard compute and our controlled lab environment, classical methods were the clear choice for this project.

3.2 Perspective Transformation

The bird’s-eye view transformation is used a lot in lane detection because it turns the perspective view (where parallel lines converge) into a top-down view (where they stay parallel). This makes the geometry much simpler to work with [6].

Mathematically, it is a homography: a 3×3 matrix $\mathbf{H}$ that maps four points in the source image to four points in the output. OpenCV gives us `cv2.getPerspectiveTransform()` to compute $\mathbf{H}$ and `cv2.warpPerspective()` to apply it [4]. One thing to keep in mind is that this only works correctly if the ground is flat — any bumps or tilts in the robot will distort the result.

The inverse $\mathbf{H}^{-1}$ is also computed in our code for projecting detected features back into the original camera frame, though in the final pipeline it is mainly used internally by the bird’s-eye view module rather than as a separate processing step.

3.3 Camera Calibration

Before we can do any reliable geometric processing, the camera needs to be calibrated. Zhang’s method [7] uses multiple photos of a checkerboard pattern taken from different angles to estimate the camera matrix $\mathbf{K}$ and the distortion coefficients. The projection model is:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (3.1)$$

Lens distortion correction is especially important for the homography step, because the homography model assumes a pinhole camera with no distortion.

3.4 Control Strategies

For line following, the three standard controllers are P, PD, and PID [8]. Each has its own strengths and weaknesses, and the choice depends heavily on the quality of the sensor data.

A proportional controller ($u = K_p \cdot e$) is the simplest: the bigger the error, the bigger the correction. It is fast, easy to implement, and has only one parameter to tune. However, a pure P controller can never fully eliminate the error in certain situations. Consider a curve: the robot needs some non-zero angular velocity to follow it, but angular velocity is proportional to error, so there must always be a non-zero error to produce that angular velocity. This is called steady-state error, and it is the fundamental limitation of proportional control. In practice, this means the robot always drives slightly off-center in curves.

Adding a derivative term gives a PD controller ($u = K_p \cdot e + K_d \cdot \dot{e}$). The derivative reacts to how fast the error is changing rather than the error itself. If the robot is approaching the lane edge quickly, the derivative produces a large corrective signal even before the error is large. This gives better transient response and reduces overshoot. But the derivative is also sensitive to noise — if the error signal is noisy (as it typically is with real cameras), the derivative amplifies that noise into jerky control signals.

PID adds an integral term that accumulates past errors over time. If the robot is consistently offset by a small amount (like in a curve), the integral slowly builds up until it produces enough correction to drive the steady-state error to zero. This sounds great in theory, but in practice the integral has a dangerous failure mode called “windup.” During a long curve, the integral keeps accumulating. When the curve ends and the error changes sign, the integral is still large from the curve, causing the robot to overshoot in the opposite direction. Anti-windup mechanisms — typically clamping the integral within a fixed range — are essential in any real PID implementation [8].

Fuzzy logic controllers are sometimes used as an alternative. They map input variables to output variables through linguistic rules like “if error is large-left and error-rate is small, then turn sharply right.” The advantage is that they do not need a mathematical model and can handle nonlinearities naturally. The disadvantage is that designing the rule base and membership functions requires a lot of expert knowledge and testing. For our well-defined problem with good mathematical models available, a classical controller seemed like the more appropriate and transparent choice.

3.5 Multi-Height Scanning

When the robot goes into a curve, one of the two lane lines often leaves the camera’s field of view. If you are only looking at one horizontal slice of the image, you lose that line completely. Scanning at multiple heights gives you redundancy: the line might be gone at one height but still visible at another. This is a practical adaptation of multi-scale processing ideas from the computer vision literature, scaled down for our specific use case.

3.6 Summary

Table 3.1 compares the three detection approaches we considered.

Table 3.1: Comparison of detection approaches for indoor line following.

Criterion	Thresholding	Canny + Hough	CNN
Computation	Very low	Low	High
Lighting sensitivity	High	Medium	Low
False positives	Low (our env.)	High	Low
Implementation effort	Low	Medium	High
TurtleBot3 suitability	High	Medium	Low

Based on all of this, we went with binary thresholding for detection, the bird's-eye view for geometry, and PID/P control depending on the environment. The next chapter takes these high-level choices and turns them into concrete requirements.

Chapter 4

Requirements Analysis

Author: Benfarhat, Mohamed Wajih

Now that we have reviewed what techniques are available, this chapter formalizes what exactly our system needs to do. We derive the requirements from the project specification, define how to test them, and set up metrics for evaluation.

4.1 Functional Requirements

Each of the seven mandatory tasks translates directly into a functional requirement.

FR-1 (Task 1): The system must calibrate the RGB camera using a checkerboard. It should output a camera matrix $\mathbf{K}$ and distortion coefficients, saved as a YAML file. We consider it successful if the reprojection error is below 1.0 pixel. The reprojection error measures how well the estimated camera model explains the observed checkerboard corners — a value below 0.5 is considered excellent, and anything above 1.0 suggests the calibration images were poor quality or too few were captured.

FR-2 (Task 2): The undistorted image must be published on `/camera/rectified_image`. We want this to use precomputed lookup tables so it does not slow down the pipeline. The undistorted image should show straight lines as straight (barrel or pincushion distortion from the lens should be removed), which is important for the homography step that comes next.

FR-3 (Task 3): The system must compute a homography matrix for the perspective transformation. The four point pairs should be configurable via YAML so we can recalibrate without touching the code. The points need to be chosen carefully: they should form a trapezoid in the camera image that maps to a rectangle in the bird's-eye view, and the aspect ratio of the output should roughly match the physical dimensions of the track area being captured.

FR-4 (Task 4): The bird's-eye view must be published on `/camera/birdseye_image` at 640×480 resolution. The output should show the track lines as approximately parallel vertical stripes, which confirms the homography is correct.

FR-5 (Task 5): LiDAR scan points from the front 120 degrees must be transformed into bird's-eye view coordinates. The coordinate transformation needs to account for the physical offset between the LiDAR sensor and the camera, as well as the scale factor between real-world meters and bird's-eye view pixels.

FR-6 (Task 6): The bird's-eye view with color-coded LiDAR overlay must be published on

`/camera/birdseye_with_lidar`. Green means close, red means far. Areas outside the camera FOV should be grayed out so that a human operator can immediately see which parts of the image contain real visual data and which parts are outside the camera’s view.

FR-7 (Task 7): The robot must detect a lane between two white lines and drive in the center. If one line disappears (e.g. in a sharp curve), it should estimate the center from the remaining line using the last known lane width. The robot should be able to do a complete lap without human help. This is the most complex requirement because it combines perception (finding the lines), estimation (computing the center), and control (driving the robot) into one integrated behavior.

4.2 Non-Functional Requirements

NFR-1: Real-time. The pipeline has to keep up with the camera frame rate. In simulation that is around 30 fps. On the real robot, we need at least 10 fps — at 0.02 m/s, a 100 ms delay means 2 mm of blind driving, which might sound small but adds up over multiple frames, especially in curves where the robot needs continuous corrections. During development we measured the actual processing time per frame: on the lab laptop, the simulation pipeline takes about 15 ms and the real robot pipeline about 25 ms, both well within the budget.

NFR-2: Robustness. The system should not crash or behave dangerously when things go wrong. If it can only see one line, it should keep going with reduced confidence. If it sees nothing for 20 frames in a row, it should stop the motors. We chose 20 frames as the threshold because at our camera frame rate that corresponds to roughly 2 seconds — long enough to not trigger during brief occlusions (a hand passing over the camera, a momentary reflection), but short enough that the robot does not drive far without any visual feedback.

NFR-3: Configurability. Controller gains, speeds, safety thresholds, and topic names should be externalized in YAML configuration files, allowing adjustment without code modification. This is especially important for the real robot, where we had to tune the angular gain and forward speed many times during testing. The binary detection thresholds (110 for simulation, 200 for real robot) are currently still hardcoded in the Python source files, which is a limitation — ideally these would also be in YAML, but in practice they only needed to be changed once per environment and then stayed fixed.

NFR-4: Dual mode. Switching between simulation and real robot should be as simple as running a different launch file. No code changes. We have three launch files: `line_follower.launch.py` for the simulation pipeline, `line_track.launch.py` for loading the Gazebo world, and `real_robot_line_follow` for the physical robot.

4.3 Use Cases

UC-1: Autonomous driving. Place the robot on the track, launch the node, and it drives laps on its own. This is the main thing the system needs to do. The preconditions are: the ROS2 workspace is built, the config files contain valid homography points for the current camera setup, and either Gazebo is running (simulation) or the TurtleBot3 is powered on and connected (real

robot). The success criterion is simple: complete a full lap without going off-track and without someone having to push or steer the robot manually.

UC-2: Live debugging. While the robot is running, a developer opens `rqt_image_view` and watches the debug topics to see what the line detector is seeing. In simulation there are four topics to watch: the rectified image, the bird’s-eye view, the LiDAR overlay, and the detection overlay. On the real robot there is just one debug topic (`/camera/line_detection`), but it shows everything we need: the scan lines, the detected clusters, and the lane center estimate. We used this use case constantly during development — sometimes we would run the robot for a lap and then go through the saved debug images frame by frame to understand why it wobbled at a particular point. This iterative debugging cycle was how we found most of the parameter values that ended up in the final config files.

UC-3: Calibration. Before the first run in a new setup, you run the interactive camera calibration tool and the homography calibration tool. The camera calibration tool shows a live view where you hold up the checkerboard, capture 10+ frames, and it computes the intrinsic parameters. The homography tool lets you click and drag four points on a live image and immediately see the bird’s-eye view update. Both tools save their results to YAML files. We designed these tools specifically because during the project we had to recalibrate multiple times — once when the camera bracket was accidentally bumped, and once when we moved the track to a different part of the lab.

4.4 Performance Metrics

We defined four metrics that we can measure from the system’s own output, without needing external equipment like a motion capture system.

M-1: Centering error (CTE). The normalized offset between the detected lane center and the image center, in the range $[-1, 1]$. Below 0.1 on straights is good. Below 0.3 in curves is acceptable.

M-2: Detection confidence. How many of the six scan heights detect both lines simultaneously. A value of 0.95 means 5–6 out of 6 heights have paired detections. Lower values mean the robot is in a curve or otherwise struggling.

M-3: Mode stability. How often the detection mode changes (centered $\rightarrow$ estimated $\rightarrow$ follow_left/right) during one lap. Lots of transitions suggest the system is right at the edge of its capability.

M-4: Lap completion. Did the robot finish a full lap without going off-track or needing a manual push? This is the ultimate pass/fail test.

4.5 Requirements Validation

We checked the requirements against Ibenthal’s four criteria. They are *correct* (each one maps to a specific task), *complete* (all seven tasks are covered plus key quality attributes), *consistent* (no contradictions — the robustness fallback adds negligible overhead, so it does not conflict with real-time performance), and *understandable* (each has a concrete acceptance criterion).

4.6 Traceability

Table 4.1 links each requirement to the module that implements it and the metric that tests it.

Table 4.1: Requirements traceability.

Req.	Module	Metric
FR-1	<code>calibrate_camera.py</code> , <code>calibration.py</code>	Reproj. error
FR-2	<code>calibration.py</code> (undistort)	Visual
FR-3	<code>birdseye.py</code> , <code>calibrate_homography.py</code>	Visual
FR-4	<code>birdseye.py</code> (transform)	Visual
FR-5	<code>lidar_transform.py</code>	Visual
FR-6	<code>line_follower_node.py</code> (overlay)	Visual
FR-7	<code>line_detector.py</code> , <code>controller.py</code>	M-1 to M-4

Chapter 5

Concept Development

Author: Sadri Ghamshi, Ashkan

With the requirements in place, this chapter explains the design decisions we made and why.

5.1 Detection: Thresholding vs. Edge Detection

We tried two approaches early on. Canny edge detection finds edges regardless of absolute brightness, but it also finds edges from shadows, floor seams, and objects near the track. The result was a noisy image full of false positives. Binary thresholding just says “everything above brightness X is a line.” Since our white tape on blue floor has excellent contrast, this works cleanly with minimal post-processing. We went with threshold 110 for simulation and 200 for the real robot.

5.2 Simulation: PID with Curvature-Aware Speed

In simulation, the sensor data is clean, so we use a full PID controller tuned to $K_p = 2.0$, $K_i = 0.02$, $K_d = 0.3$. A dead zone of ± 0.05 prevents jittery steering when centered. Speed adapts based on curvature: sharp curves get 40% of the base speed (0.12 m/s), medium curves 60%, gentle curves 80%, and straights get full speed.

5.3 Real Robot: P-Only with Multi-Height Scanning

When we first tried PID on the real robot, it was a disaster. The integral term accumulated noise-induced error and caused sudden jerks; the derivative amplified camera vibration into erratic steering. After two full lab sessions trying different gains, we concluded the noise floor is simply too high for I and D terms. So we went with $\omega = -\text{offset} \times 0.2$, capped at ± 0.3 rad/s, at 0.02 m/s.

To compensate, we made the perception smarter with six horizontal scan lines at different heights (25% to 75%). If a line exits the field of view at one height during a curve, it might still be visible at another.

5.4 Fallback Strategy

We implemented a three-tier fallback. In *centered* mode, both lines are visible at the same scan height. In *estimated* mode, both lines are visible but at different heights. In *follow* mode, only one line is visible and we offset 150 pixels from it. If nothing is visible for 20 frames, the robot stops.

Chapter 6

Implementation

Author: Sadri Ghamshi, Ashkan

This chapter walks through the actual code and algorithms, module by module.

6.1 Camera Calibration and Undistortion

The calibration tool (`calibrate_camera.py`) is an interactive ROS2 node. You hold an 8×6 checkerboard (2.5 cm squares) in front of the camera, press space to capture frames from different angles, and after at least 10 captures, press C to run calibration via `cv2.calibrateCamera()` (Zhang’s method [7]). It outputs the camera matrix **K** and distortion coefficients, saved to YAML. At runtime, `calibration.py` precomputes undistortion lookup tables with `cv2.initUndistortRectifyMap()` making undistortion a single fast `cv2.remap()` call per frame. Note: the calibration file did not end up in the final repository; the code handles this with a fallback warning.

6.2 Bird’s-Eye View and LiDAR Overlay

The homography module (`birdseye.py`) loads four source/destination point pairs from YAML and computes the transformation with `cv2.getPerspectiveTransform()`. An interactive tool (`calibrate_homography.py`) lets you drag points and see the result live. The LiDAR module converts scan measurements from polar to pixel coordinates in the bird’s-eye view, filtering to the front 120 degrees and color-coding by distance (green = close, red = far).

6.3 Line Detection — Simulation

The simulation detector works on the bird’s-eye view: grayscale conversion, 5×5 Gaussian blur, threshold at 110, then row-by-row scanning from bottom upward (every 5 rows). The median of collected positions gives robust estimates x_L and x_R . The lane center is:

$$x_{\text{center}} = \begin{cases} (x_L + x_R)/2 & \text{both lines visible} \\ x_L + 50 & \text{only left line} \\ x_R - 50 & \text{only right line} \end{cases} \quad (6.1)$$

Curvature is estimated by comparing line positions in the top and bottom halves ($R \approx |500/\Delta x|$), and a 10-frame moving average smooths the output.

6.4 Line Detection — Real Robot

The real robot detector had to be written from scratch. The raw 800×600 image (rotated 180°) is cropped to the bottom half, resized to 640×480 , thresholded at 200, and cleaned with morphological operations: 3×3 erosion removes noise, 5×5 dilation restores line thickness. Connected components under 1000 pixels are discarded.

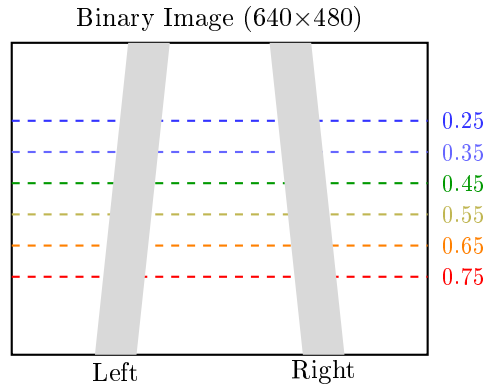


Figure 6.1: Multi-height scanning: six lines from far (blue) to near (red).

Six scan heights (0.25 to 0.75) each look at five adjacent rows. White pixels are clustered with a 5-pixel gap tolerance and minimum size of 3. The fallback picks paired detections (80% nearest, 20% farthest), unpaired estimates, or single-line guesses offset by 150 pixels.

6.5 Controller Design

6.5.1 Simulation (PID)

The PID formula is $\omega = -(K_p e + K_i \int e dt + K_d \dot{e})$ with $K_p = 2.0$, $K_i = 0.02$, $K_d = 0.3$. A dead zone at ± 0.05 prevents unnecessary corrections when centered. The integral is clamped to $[-1, 1]$ to prevent windup, and angular velocity is limited to $\pm 1.5 \text{ rad/s}$. Speed adapts to curvature as described in Chapter 5: the system compares line positions in the upper and lower halves of the bird's-eye view to estimate how sharp the upcoming curve is, then scales speed from 40% (sharp) to 100% (straight) of the base 0.12 m/s.

6.5.2 Real Robot (P-Only)

The real robot uses $\omega = -\text{offset} \times 0.2$, capped at $\pm 0.3 \text{ rad/s}$. Forward speed is a constant 0.02 m/s, halved when the offset exceeds 0.3 to give the controller more time in tight curves. The system stops after 20 consecutive frames with no line detection. The simplicity of this controller is deliberate: on the real robot, the noise in the offset signal is too high for derivative or integral terms to be useful without dedicated filtering.

Table 6.1: Controller comparison.

Parameter	Simulation	Real Robot
Type	PID	P-only
Base speed	0.12 m/s	0.02 m/s
Max angular	1.5 rad/s	0.3 rad/s
Speed adaptation	4-level	Binary (100%/50%)

Chapter 7

Testing and Evaluation

Author: Sadri Ghamshi, Ashkan

This chapter covers how well the system performs in simulation and on the real robot.

7.1 Simulation Results

On straight parts, the robot keeps a centering error below 0.03, well within target. The PID controller with curvature-aware speed handles corners smoothly: the CTE rises to roughly ± 0.15 in the sharpest turns, then settles back below 0.05 within about 10 frames. Full laps complete consistently.

7.2 Real Robot Results

We analyzed 36 debug images from one test run. Each image shows the six scan lines, detected clusters, and the computed lane center.

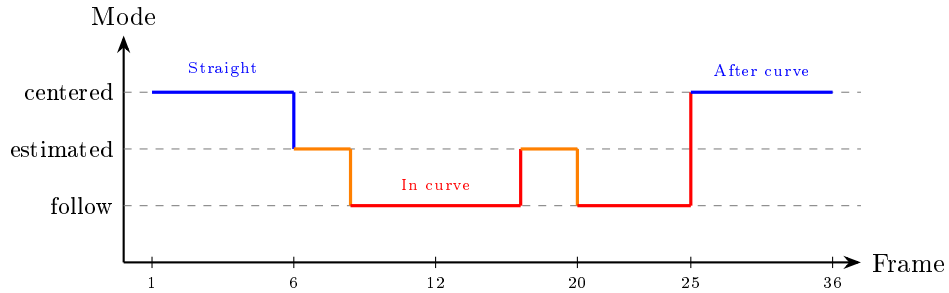


Figure 7.1: Detection mode transitions through a curve.

On the straight, we are in centered mode with CTE around ± 0.27 . Once the curve starts (frame 6), the outer line disappears at far scan heights and we drop to estimated mode. During the turn (frames 9–20), the system bounces between follow and estimated. Around frame 15, the offset crosses 0.3 and speed halves to 0.01 m/s. After the curve we recover to centered; the final offset settles around 0.094 due to the P controller’s inherent steady-state error.

7.3 Performance Summary

Table 7.1: Performance comparison.

Metric	Simulation	Real Robot
M-1: CTE (straight)	< 0.03	± 0.27
M-1: CTE (curve)	± 0.15	± 0.60
M-2: Confidence	1.0	0.95
M-3: Mode transitions/lap	0	15–25
M-4: Lap completion	Yes	Yes

The simulation is about $5\times$ more accurate, but gets perfect sensor data. What matters is that both modes complete the lap.

7.4 Issues

The calibration file was not committed to git. The blue floor causes specular reflections that trigger false detections — the erosion/dilation pipeline handles most of it but is not a clean solution.

Chapter 8

Conclusion

Author: Sadri Ghamshi, Ashkan

8.1 What We Built

Over the course of this project, we built a complete lane-following system for the TurtleBot3 Waffle that handles all seven mandatory tasks. The pipeline goes from raw camera input through calibration, bird’s-eye view transformation, LiDAR overlay, line detection, and velocity control. It runs in two modes: simulation with PID control at 0.12 m/s and real robot with proportional control at 0.02 m/s. Both modes complete full laps autonomously.

8.2 What Worked Well

The multi-height scanning was the most important design decision for the real robot. Without six scan lines, any curve would be a disaster because the outer line disappears at certain heights. The three-tier fallback (centered/estimated/follow) proved its value by letting the robot gracefully degrade instead of stopping. On the simulation side, curvature-aware speed control made a noticeable difference in corner handling.

8.3 What Did Not Go So Well

The biggest headache was the sim-to-real transfer. Our PID controller worked beautifully in simulation but was unusable on the real robot due to noise. The missing calibration file is a workflow failure. The blue floor reflections remain an open problem.

8.4 Future Improvements

A small integral term ($K_i \approx 0.005$) could reduce steady-state error without windup. Otsu’s method would handle lighting changes. Implementing bird’s-eye view on the real robot would unify both pipelines. A Kalman filter could smooth the offset signal better than a moving average.

Bibliography

- [1] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*. Springer, 2nd ed., 2016.
- [2] ROBOTIS, *TurtleBot3 e-Manual*, 2023.
- [3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” in *Science Robotics*, vol. 7, 2022.
- [4] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [5] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. Springer, 2nd ed., 2017.
- [6] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd ed., 2003.
- [7] Z. Zhang, “A flexible new technique for camera calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [8] K. Ogata, *Modern Control Engineering*. Prentice Hall, 5th ed., 2010.

Appendix A

Appendix

A.1 Configuration Files

A.1.1 Real Robot Parameters

The following YAML file (`config/real_robot_params.yaml`) contains all tunable parameters for the real robot mode:

```
1 # Real Robot Line Follower Configuration
2 # TurtleBot3 Waffle - Continuous proportional control
3 control:
4   forward_speed: 0.02      # m/s constant forward speed
5   angular_gain: 0.2        # steering gain
6   max_angular: 0.3         # max angular velocity (rad/s)
7
8 safety:
9   max_no_line_count: 20    # frames without line before stop
10
11 debug:
12   publish_debug_images: true
13   log_frequency: 30        # log every N frames
14   save_interval: 30        # save debug image every N frames
15
16 camera:
17   topic: "/camera/image_raw"
18   qos_depth: 20
19
20 topics:
21   cmd_vel: "/cmd_vel"
22   camera_detection: "/camera/line_detection"
```

Listing A.1: Real robot configuration parameters.

A.1.2 Simulation Homography Points

The homography transformation for the simulation uses the following source and destination points stored in `config/homography.yaml`:

```

1 source_points:
2   - [540, 380]      # top-left of trapezoid
3   - [1380, 380]     # top-right of trapezoid
4   - [1920, 800]     # bottom-right
5   - [0, 800]        # bottom-left
6
7 destination_points:
8   - [0, 0]           # top-left of rectangle
9   - [640, 0]         # top-right
10  - [640, 480]        # bottom-right
11  - [0, 480]          # bottom-left

```

Listing A.2: Simulation homography configuration.

A.2 Project Repository Structure

```

1 hawk-robotics-project/
2   src/
3     line_follower_pkg/
4       config/
5         homography.yaml
6         real_robot_params.yaml
7         real_robot_homography.yaml
8       launch/
9         line_follower.launch.py
10        line_track.launch.py
11        real_robot_line_follower.launch.py
12      line_follower_pkg/
13        __init__.py
14        line_follower_node.py
15        calibration.py
16        birdseye.py
17        line_detector.py
18        controller.py
19        lidar_transform.py
20        calibrate_homography.py
21        calibrate_camera.py
22      real_robot/
23        __init__.py
24        real_robot_line_follower_node.py
25        real_line_detector.py
26        real_robot_controller.py
27      models/

```

```
28     track_ground/model.sdf
29     curved_track/model.sdf
30     white_line/model.sdf
31 worlds/
32     line_track.sdf
33     simple_lines.sdf
34 package.xml
35 setup.py
```

Listing A.3: Directory structure of the project repository.